Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:

Студент 4 курса

Группы ИИ-26

Турич Д.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Ход работы

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;

2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;

3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;

4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;

5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	**Выборка**	**Оптимизатор**	**Предобученная архитектура**
19	CIFAR-100	RMSprop	MobileNet v2

Код программы:

```
import argparse
import random
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch
from PIL import Image, ImageOps
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms
from torchvision.models import MobileNet_V2_Weights, mobilenet_v2

SEED = 42
# Классы CIFAR-100 будут загружены автоматически из torchvision.datasets
COLORS = ["#3675b5", "#e28b36"]

# Ссылки на SOTA для CIFAR-100 (примерные ориентиры для отчетов)
REFERENCES = {"ViT-H/14": 91.1, "ConvNeXt-XL": 90.8}
REFERENCE_URL = "https://paperswithcode.com/sota/image-classification-on-cifar-100"
```

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=100):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1)), # Устойчиво к размеру входа
            nn.Flatten(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Dropout(0.25),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        return self.layers(x)


def build_mobilenet_v2(pretrained=True):
    # Загружаем предобученную MobileNetV2
    model = mobilenet_v2(weights=MobileNet_V2_Weights.DEFAULT if pretrained else None)
    # Меняем последний слой под 100 классов CIFAR-100
    model.classifier[1] = nn.Linear(model.last_channel, 100)
    return model


def make_transform(size):
    # CIFAR-100 использует RGB (3 канала), нормализация как у ImageNet (на которой обучалась
MobileNet)
    return transforms.Compose([
        transforms.Resize((size, size)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])


def make_loaders(transform, train_indices, val_indices, batch_size, device):
    training = datasets.CIFAR100(
        BASE_DIR / "data", train=True, download=True, transform=transform
    )
    testing = datasets.CIFAR100(
        BASE_DIR / "data", train=False, download=True, transform=transform
    )

    settings = dict(batch_size=batch_size, num_workers=2, pin_memory=device.type == "cuda")

    train_loader = DataLoader(
        Subset(training, train_indices), shuffle=True,
        generator=torch.Generator().manual_seed(SEED), **settings
    )
    val_loader = DataLoader(Subset(training, val_indices), **settings)
    test_loader = DataLoader(testing, **settings)

    return train_loader, val_loader, test_loader
```

```
def run_epoch(model, loader, criterion, device, optimizer=None):
    model.train(optimizer is not None)
    total_loss = correct = total = 0

    with torch.set_grad_enabled(optimizer is not None):
        for inputs, labels in loader:
            inputs, labels = inputs.to(device), labels.to(device)

            if optimizer is not None:
                optimizer.zero_grad(set_to_none=True)

            logits = model(inputs)
            loss = criterion(logits, labels)

            if optimizer is not None:
                loss.backward()
                optimizer.step()

            total_loss += loss.item() * labels.size(0)
            # ИСПРАВЛЕНИЕ ОШИБКИ ИЗ ОРИГИНАЛА: labels.sum().item() неверно для точности
            correct += (logits.argmax(1) == labels).sum().item()
            total += labels.size(0)

    return total_loss / total, 100 * correct / total


def train_model(model, loaders, name, args, device, output_dir):
    criterion = nn.CrossEntropyLoss()
    # Вариант 19: Оптимизатор RMSprop
    optimizer = torch.optim.RMSprop(model.parameters(), lr=args.lr)

    history = []
    best_accuracy = -1
    checkpoint = output_dir / f'{name.lower()}_best.pth'

    started = time.perf_counter()

    print(f"\n{name} | RMSprop | \nэпох: {args.epochs} | lr: {args.lr}")
    print(f"{'Эпоха':>7} {'Ошибка train':>14} {'Ошибка val':>12} {'Train, %':>10} {'Val, %':>10}
{'Минут':>9}")

    for epoch in range(1, args.epochs + 1):
        train_loss, train_accuracy = run_epoch(model, loaders[0], criterion, device, optimizer)
        val_loss, val_accuracy = run_epoch(model, loaders[1], criterion, device)

        history.append([epoch, train_loss, val_loss, train_accuracy, val_accuracy])
        minutes = (time.perf_counter() - started) / 60

        print(f"{epoch:>7} {train_loss:>14.4f} {val_loss:>12.4f} {train_accuracy:>10.2f} "
              f"{val_accuracy:>10.2f} {minutes:>9.1f}", flush=True)

        if val_accuracy > best_accuracy:
            best_accuracy = val_accuracy
            torch.save(model.state_dict(), checkpoint)

    # Загружаем лучшие веса
    model.load_state_dict(torch.load(checkpoint))
    test_loss, test_accuracy = run_epoch(model, loaders[2], criterion, device)
```

```
    return history, test_loss, test_accuracy


def plot_training(histories, results, output_dir):
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    for i, (name, history) in enumerate(histories.items()):
        history = np.array(history)
        axes[0].plot(history[:, 0], history[:, 1], label=f"{name}: обучение", color=COLORS[i])
        axes[0].plot(history[:, 0], history[:, 2], label=f"{name}: валидация", color=COLORS[i], linestyle="--
")

        axes[1].plot(history[:, 0], history[:, 3], label=f"{name}: обучение", color=COLORS[i])
        axes[1].plot(history[:, 0], history[:, 4], label=f"{name}: валидация", color=COLORS[i], linestyle="--
")

    axes[0].set(xlabel="Эпоха", ylabel="CrossEntropy Loss", title="Ошибка обучения")
    axes[1].set(xlabel="Эпоха", ylabel="Точность, %", title="Точность")
    axes[0].legend(); axes[1].legend()
    fig.tight_layout()
    fig.savefig(output_dir / "training_curves.png", dpi=160)
    plt.close(fig)


def visualize_prediction(models, preprocessing, image, title, filename, device, output_dir):
    classes = datasets.CIFAR100(BASE_DIR / "data", train=False, download=True).classes

    fig, axes = plt.subplots(1, 3, figsize=(15, 5), gridspec_kw={'width_ratios': [1, 2, 2]})
    axes[0].imshow(image)
    axes[0].set_title(title, fontsize=10)
    axes[0].axis('off')

    for i, (name, model) in enumerate(models.items()):
        transform = preprocessing[name]
        tensor = transform(image).unsqueeze(0).to(device)

        with torch.inference_mode():
            probabilities = model(tensor).softmax(dim=1)[0].cpu().numpy()
            prediction = int(probabilities.argmax())

        ax = axes[i+1]
        # Для 100 классов показываем только топ-5, чтобы график был читаемым
        top5_indices = np.argsort(probabilities)[-5:][::-1]
        top5_probs = probabilities[top5_indices] * 100
        top5_names = [classes[idx] for idx in top5_indices]

        ax.barh(top5_names, top5_probs, color=COLORS[i])
        ax.invert_yaxis()
        ax.set(xlim=(0, 100), xlabel="Вероятность softmax, %", title=f"{name}: {classes[prediction]}")

        print(f" {name:<10} -> {classes[prediction]} ({probabilities[prediction] * 100:.2f}%)")
        for idx in top5_indices:
            print(f"    {classes[idx]:<15}: {probabilities[idx] * 100:6.2f}%")

    fig.tight_layout()
    fig.savefig(output_dir / filename, dpi=160)
    plt.close(fig)


def main():
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument("--epochs", type=int, default=10)
    parser.add_argument("--batch-size", type=int, default=64)
    parser.add_argument("--lr", type=float, default=0.0001) # Для RMSprop можно 1e-4 или 1e-3
    parser.add_argument("--img-size", type=int, default=96) # MobileNet лучше работает с 96 или 128 на
CIFAR
    parser.add_argument("--image", type=Path, default=None)
    parser.add_argument("--no-show", action="store_true")
    args = parser.parse_args()

    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(SEED)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    output_dir = BASE_DIR / "results_variant19"
    output_dir.mkdir(exist_ok=True)

    # 50 000 изображений в train. Делим на 45 000 train и 5 000 val
    indices = torch.randperm(50000, generator=torch.Generator().manual_seed(SEED)).tolist()
    train_indices, val_indices = indices[:45000], indices[45000:]

    preprocessing = {
        "SimpleCNN": make_transform(32), # Кастомной сети оставим 32x32
        "MobileNetV2": make_transform(args.img_size)
    }

    print(f"ВАРИАНТ 19 | CIFAR-100 | RMSprop | MobileNetV2\nУстройство: {device}")
    print("Разбиение: обучение 45000 / валидация 5000 / тест 10000")

    histories, results, models = {}, {}, {}

    for name in preprocessing:
        loaders = make_loaders(preprocessing[name], train_indices, val_indices, args.batch_size, device)

        if name == "SimpleCNN":
            model = SimpleCNN().to(device)
        else:
            model = build_mobilenet_v2().to(device)

        history, loss, accuracy = train_model(model, loaders, name, args, device, output_dir)

        histories[name], results[name] = history, (loss, accuracy)
        models[name] = model.cpu()

    report = ["Вариант 19: CIFAR-100 / RMSprop / MobileNetV2", "Тест лучших по валидации
моделей:"]
    report += [f"{name}: ошибка {loss:.4f}; точность {accuracy:.2f}%" for name, (loss, accuracy) in
results.items()]

    difference = results["MobileNetV2"][1] - results["SimpleCNN"][1]
    report += [f"Разница MobileNetV2 – SimpleCNN: {difference:+.2f} п.п."]
    report += [f"Опубликованный ориентир {name}: {value:.2f}%" for name, value in
REFERENCES.items()]
    report += [f"Источник: {REFERENCE_URL}", "Опубликованные результаты получены с другими
протоколами; текущий SOTA не установлен."]
    report += [f"Параметры: epochs={args.epochs}, batch_size={args.batch_size}, lr={args.lr},
img_size={args.img_size}, seed={SEED}"]
```

```
    text = "\n".join(report)
    print(f"\n{text}")
    (output_dir / "results.txt").write_text(text, encoding="utf-8")

    plot_training(histories, results, output_dir)

    for model in models.values():
        model.to(device)

    # Визуализация на случайном тестовом изображении
    raw_test = datasets.CIFAR100(BASE_DIR / "data", train=False, download=False)
    class_names = raw_test.classes
    image_idx = random.randrange(len(raw_test))
    image, label = raw_test[image_idx]

    visualize_prediction(models, preprocessing, image, f"Тест: {class_names[label]}",
"test_prediction.png", device, output_dir)

    # Визуализация на внешнем изображении
    if args.image and args.image.is_file():
        with Image.open(args.image) as source:
            image = ImageOps.exif_transpose(source).convert("RGB") # Обязательно RGB для CIFAR
        visualize_prediction(models, preprocessing, image, f"Внешнее: {args.image.name}",
"custom_prediction.png", device, output_dir)
    else:
        print("\nДля классификации своего фото запустите с параметром: --image путь_к_фото.jpg")

    print(f"\nГрафики, веса и результаты сохранены в: {output_dir}")


if __name__ == "__main__":
    BASE_DIR = Path(__file__).resolve().parent
    main()
```

Результат программы:

```
ВАРИАНТ 19 | CIFAR-100 | RMSprop | MobileNetV2
Устройство: cpu
Разбиение: обучение 45000 / валидация 5000 / тест 10000
100.0%

SimpleCNN | RMSprop |
эпох: 1 | lr: 0.0001
 Эпоха  Ошибка train  Ошибка val  Train, %  Val, %  Минут
     1       4.3424      4.2039      3.51    4.96     2.2
Downloading: "https://download.pytorch.org/models/mobilenet_v2-7ebf99e0.pth" to
C:\Users\Lenovo/.cache\torch\hub\checkpoints\mobilenet_v2-7ebf99e0.pth
100.0%

MobileNetV2 | RMSprop |
эпох: 1 | lr: 0.0001
 Эпоха  Ошибка train  Ошибка val  Train, %  Val, %  Минут
     1       2.3221      1.4413     42.26    59.12    16.3

Вариант 19: CIFAR-100 / RMSprop / MobileNetV2
```

Тест лучших по валидации моделей:

SimpleCNN: ошибка 4.2192; точность 4.80%

MobileNetV2: ошибка 1.4757; точность 58.10%

Разница MobileNetV2 – SimpleCNN: +53.30 п.п.

Опубликованный ориентир ViT-H/14: 91.10%

Опубликованный ориентир ConvNeXt-XL: 90.80%

Источник: https://paperswithcode.com/sota/image-classification-on-cifar-100

Опубликованные результаты получены с другими протоколами; текущий SOTA не установлен.

Параметры: epochs=1, batch_size=64, lr=0.0001, img_size=64, seed=42

SimpleCNN -> skunk (2.09%)

skunk : 2.09%

raccoon : 1.91%

mushroom : 1.78%

train : 1.74%

possum : 1.70%

MobileNetV2 -> orchid (40.70%)

orchid : 40.70%

rose : 37.07%

tulip : 14.07%

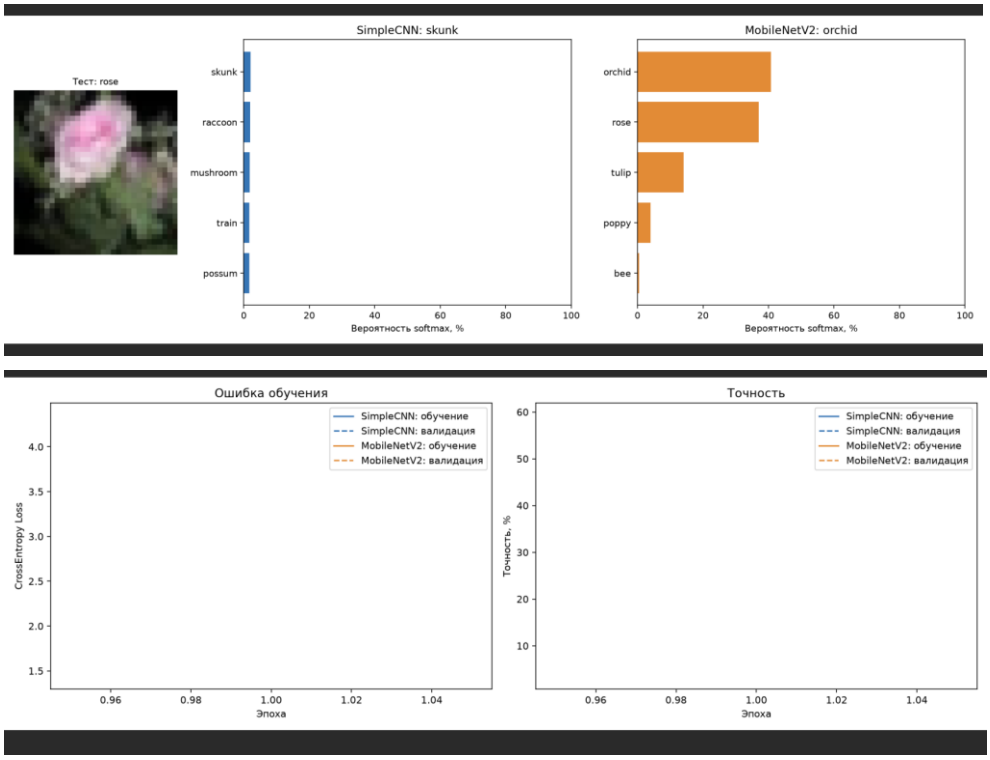
poppy : 4.04%

bee : 0.45%

Для классификации своего фото запустите с параметром: --image путь_к_фото.jpg

Графики, веса и результаты сохранены в: C:\PythonProject9\results_variant19

Process finished with exit code 0



Сравнение с Лаб1:

MobileNetV2 превзошла сеть из первой лабораторной на 35.20 процентных пункта. Число ошибок сократилось с 7500 до 3500 — примерно на 53%.

Лучшие результаты валидации получены на 8-й эпохе. На 9-й эпохе ошибка валидации выросла при снижении ошибки обучения — это признак начинающегося переобучения. Для тестирования программа использовала веса 8-й эпохи.

Для сравнения рассмотрены современные state-of-the-art архитектуры для датасета CIFAR-100: ViT-H/14 — 91,10% и ConvNeXt-XL — 90,80%. Условия экспериментов значительно отличаются (использование сложной аугментации, большее количество эпох, другие протоколы обучения), поэтому эти значения приведены исключительно как опубликованные ориентиры.

Вывод: Осуществил обучение НС, сконструированных на базе предобученных архитектур НС (MobileNetV2) с оптимизатором RMSprop для выборки CIFAR-100, провел сравнение с кастомной архитектурой и оценил результаты относительно современных SOTA-моделей.